The expression `+$a` converts `$a` to a numeric type (if possible, the usual type conversion rules apply), and `-$num` will make a positive number negative. Moving beyond the basics, we can work with exponents (calculating $x^n$) and perform modular arithmetic (calculating the remainder after division):

```
var_dump(2 ** 16);
var_dump(10 % 3);
```

```
Output:
int(65536)
int(1)
```

`2 ** 16` gives us the sixteenth power of *2*, which is *65536*, and *10 / 3* is *3* with a remainder of *1*.

Finally, there are operators for C-style increment and decrement: `++` and `–`, respectively:

```
$a = 1;
var_dump($a++);
var_dump($a);
var_dump(++$a);
```

```
Output:
int(1)
int(2)
int(3)
```

From the above we can see that `$a++` increments the value after it has been evaluated in the expression (called *post-increment*), then on the final line, we use `++$a` to increment before evaluation (*pre-increment*). Exchanging `++` for `--` in the code above would adjust values by *-1* but otherwise work the same way.

Using just this handful of operators, it is possible to construct all sorts of wonderful equations, but for the sake of convenience and your sanity, PHP comes loaded with many built-in functions, which we will be reviewing shortly.